Shape-Aware Sparse Traffic-Matrix Analytics with Conformal Identifiability Screens for GraphChallenge Network Sensing

Qifan Chen
University of California, Santa Barbara
qifan_chen@ucsb.edu

***Abstract*—The MIT/IEEE/Amazon GraphChallenge Anonymized Network Sensing track asks participants to construct and analyze anonymized source-to-destination traffic matrices from packet-derived data. We present a commodity-CPU toolkit whose core method is certified, memory-priced identifiability for sketch-based heavy-hitter sensing: decide whether top-k heavy hitters are identifiable under a sketch memory budget; if so certify it, and if not, price the memory required or declare the task unidentifiable. The method builds on an empirical finding with a sparse-recovery explanation – candidate-tracking capacity, not Count-Min width, bounds discovery on diffuse traffic, a gradient two decoder families reproduce – and on *conformal identifiability screens*: window-level calibration lower-bounds future ranking gaps under exchangeability, replacing single-prefix representativeness with finite-sample marginal coverage (adaptive conformal inference empirically restores target miscoverage under the tested drift settings). The pricing algorithm refuses a 655 KB budget a single-prefix screen had accepted, prices certification at 11.7 MB on a synthetic hotspot regime and 16.8 MB on real CTU-13 traffic – with measured recall 1.0 at the priced budgets – and declares the all-tie official Random PCAP unidentifiable, where a budget-matched identifiability-aware recall replaces a misleading zero. A certified router composes these components into one operational decision per stream: exact path, sketch certification with its price, backend, and safe fallbacks, abstaining when its signals disagree. We validate against the official challenge products: 40,000,000 packets parsed in seconds and official GraphBLAS matrices reproduced exactly (64 of 64 windows). The contribution is not full-capture throughput, but a reproducible bridge between official GraphChallenge materials and certified commodity sparse-matrix experimentation.**

I. Introduction

The Anonymized Network Sensing Graph Challenge frames packet-derived records as sparse source-to-destination traffic matrices for graph and linear-algebra analysis [1]; submissions are paper-based and reward measured innovation across software, hardware, algorithms, and systems [2], [3]. This submission targets a complementary point in that design space: not maximum throughput, but a reproducible, low-dependency baseline that makes traffic-shape dependence and approximation risk measurable – and certifiable.

To be explicit about what kind of paper this is: it is a diagnostics-and-certification contribution, not a performance record. The central empirical finding is that candidate-tracking capacity, not Count-Min sketch width, bounds heavy-hitter discovery on diffuse traffic; a sparse-recovery reading (width buys measurements, capacity buys decoding) explains it and predicts that it is decoder-independent, which we verify. The contributions are layered around that finding. The *certification layer* – the primary contribution – turns traffic-shape measurements into guarantees: a sufficient-condition sketch-safety screen with a SpaceSaving retention bound that quantitatively predicts the required capacity, a single identifiability index I_k with a budget-matched identifiability-aware recall metric, *conformal identifiability screens* that replace the prefix-representativeness assumption with finite-sample marginal coverage over stream windows (adaptive conformal inference empirically restores target miscoverage under the tested drift settings), and a *budget-pricing algorithm* that outputs the memory-minimal certified sketch configuration or declares the stream unidentifiable. The *validation layer* supports the diagnostics with multi-regime synthetic benchmarks over equivalence-checked exact baselines, a real capture (CTU-13), official Random PCAP prefixes, an exact cross-check against the official GraphBLAS matrices, and a SuiteSparse:GraphBLAS baseline. The *toolkit layer* – capability, not headline – adds an early-stream selector, time-bin anomaly summaries, format checks, and scripts regenerating every table and figure here. All components compose into a *certified router* (Algorithm 2), and the central experimental artifact is the identifiability phase diagram of Figure 1: across sources, budgets, and two decoder families, no certified point ($I_k \geq 1$) has low recall.

II. Related Work and Positioning

GraphChallenge traffic-matrix tasks align with sparse linear algebra: GraphBLAS defines graph analytics as sparse matrix operations [4], SuiteSparse:GraphBLAS implements them efficiently [5], and reference-implementation work improves the official pipeline [6]. Ours is complementary: a certification-first CPU toolkit, not a replacement for high-performance reference code.

Network measurement has long used sketches and heavy-hitter algorithms for high-rate streams: Count-Min [7], Misra-Gries [8], SpaceSaving [9], and sketching systems [10]. Sketches are linear measurements of a sparse vector, so heavy-hitter discovery is a sparse-recovery problem [11]. On the uncertainty side, conformal prediction gives distribution-free finite-sample guarantees [12], extends to drifting data [13], and has been applied to frequency estimation from sketches [14]; we conformalize a different object – the top-k identifiability decision – rather than per-query frequency intervals. Traffic-anomaly work shows feature distributions matter as much as volume [15].

A. Student Innovation

The student innovation is not a hardware record but a certification-first reproducibility package for commodity CPUs, intentionally not optimized with GPU, C++, or distributed GraphBLAS – a baseline with guarantees rather than a performance ceiling.

III. Approach

The input is an anonymized edge table (source, destination, numeric weight such as bytes or packets). The pipeline factorizes labels, builds a SciPy coordinate matrix, converts to compressed sparse row, and sums duplicates, yielding $A \in \mathbb{R}^{m \times n}$ with A_{ij} the aggregate traffic from source i to destination j.

The implementation computes network-sensing analytics: size, nonzeros, density, total traffic, top-k pairs, degree and strength distributions, strength and edge-weight entropies, a leading singular value, and an edge-weight Gini coefficient.

Three exact construction methods – direct sparse, pandas groupby, and a pure Python dictionary baseline – are checked for output equivalence on the same inputs. The streaming path keeps exact strength summaries, a fixed-size Count-Min Sketch [7], and a bounded SpaceSaving-style candidate tracker [8] with lazy-deletion heap eviction, so even all-unique streams cost $O(\log c)$ per record; its main diagnostic is top-k recall against the exact matrix, isolating candidate discovery from weight estimation.

A deterministic synthetic generator provides four regimes (`hotspot_zipf`, `community_bursty`, `scanner_fanout`, `uniform_sparse`) with controlled knobs for size, duplication, skew, community structure, fanout, and bursts; it is a reproducibility tool, not a validated traffic model. As a toolkit capability, a time-bin path computes per-bin shape statistics and a robust anomaly score.

Finally, the online early-stream selector uses only a stream prefix to compute duplication rate, estimated nonzero growth, source and destination entropy, edge Gini, and top-share concentration, recommending pandas pre-aggregation when the prefix is concentrated and sparse direct construction otherwise, with fixed-candidate sketching marked unsafe for diffuse prefixes. Its thresholds are empirical and chosen only to separate the synthetic regimes used here; Section VI reports a real-traffic miss, which is why the certification layer, not this heuristic, is the primary contribution.

IV. Conservative Screening Conditions

The safety screen adds conservative sufficient conditions to the empirical selector rule; it is a prefix-computed screen, not a guarantee about the full stream. The candidate tracker is SpaceSaving with weighted updates [9]: an arriving tracked key adds its weight; an untracked key replaces a minimum-count key and inherits that count plus its weight. By the standard SpaceSaving/Misra-Gries analysis [8], [9], every tracked count overestimates its key's true weight by at most $W/(c+1)$ for stream weight W and capacity c, and every key with true weight above $W/(c+1)$ is tracked. Let w_k be the kth largest aggregated prefix edge weight and $\Delta_k = w_k - w_{k+1}$ the ranking gap; we define top-k identity only when $\Delta_k > 0$, and the screen refuses when $\Delta_k = 0$ (ties), as on the official prefix below.

The screen assumes: (A1) *prefix representativeness* – weight shares and rank order measured on the prefix carry over to the stream (checked empirically across eight disjoint official-capture segments in Section VI, but not guaranteed); (A2) Count-Min with width m, depth d overestimates any queried weight by at most ϵW, $\epsilon = e/m$, with probability at least $1 - e^{-d}$ per query [7]. Under (A1)–(A2), three sufficient conditions follow. *Retention*: $c \geq W/w_k - 1$ ensures every true top-k pair is tracked. *Ranking*: $\Delta_k > 2\epsilon W$ ensures Count-Min estimates preserve the top-k boundary among tracked keys. *Combined*: the implemented screen requires the stricter $\Delta_k > 2(\epsilon W + W/(c+1))$, which additionally covers rankings read from tracker counts rather than sketch estimates. Failure of the screen does not prove sketch failure; satisfaction certifies top-k retention and separation under the stated assumptions only.

A. A Sparse-Recovery View and Identifiability

Count-Min is a linear sketch with a sparse binary measurement matrix, and heavy-hitter discovery is a sparse-recovery problem [11]: sketch width buys measurements (it controls estimation noise ϵW), while candidate capacity or tree-search budget is the *decoder*. This view predicts the central finding of Section VI rather than merely describing it: on concentrated traffic the aggregated weight vector is effectively sparse and any reasonable decoder recovers it, while all-tie traffic is a maximally flat signal that no decoder can identify from any number of measurements, because top-k membership carries no information. We summarize a stream-budget pair by a single dimensionless *top-k identifiability index*

$$I_k = \frac{\Delta_k}{2\left(\epsilon W + W/(c+1)\right)}, \tag{1}$$

so that $I_k \geq 1$ is exactly the combined screen condition. Because $w_k \geq \Delta_k$ always, $I_k \geq 1$ implies the retention condition as well, so one number decides the certificate.

The same error budget fixes the evaluation metric. With budget-matched tolerance $\gamma = 2(\epsilon + 1/(c+1))$, define the *identifiable core* $H_k^\gamma = \{x \in \text{top-}k : (w_x - w_{k+1})/W > \gamma\}$ – the top-k pairs separated from the boundary by more than the budget's error resolution – and *IdentifiableRecall@k* as recall restricted to H_k^γ, reporting *not identifiable* when the core is empty rather than a misleading zero. By construction the core is empty exactly when the screen refuses, so metric and screen cannot disagree. Section VI tests the decoder-independence prediction directly by comparing SpaceSaving tracking against dyadic Count-Min tree descent [7]. The dyadic decoder treats the key space as a binary tree: it maintains one Count-Min sketch per prefix length, queries estimates for dyadic intervals level by level, expands only the highest-mass intervals, and continues until reaching individual keys. Unlike SpaceSaving it stores no per-record candidates; its discovery budget is the number of interval expansions per level. It therefore provides an independent decoder family for testing whether diffuse-regime failure is an artifact of candidate tracking or intrinsic to the signal.

B. Conformal Identifiability Screens

The plug-in screen above trusts one prefix (A1). We remove that assumption with conformal prediction [12]: split the stream into disjoint equal windows, treat them as exchangeable, and score each by its gap share $S_i = \Delta_k^{(i)}/W^{(i)}$. With n calibration windows and $r = \lfloor \alpha(n{+}1) \rfloor \geq 1$, let L be the rth smallest calibration score; the order-statistic argument gives $\Pr[S_{\text{future}} < L] \leq \alpha$.

Proposition 1 (window-level identifiability screen). Assume the calibration and future window scores are exchangeable, and condition on the Count-Min per-query error event of (A2). If $L > 2(\epsilon + 1/(c{+}1))$, then with probability at least $1 - \alpha$ over the future window, its true top-k pairs are all retained by the tracker and correctly separated from non-top-k pairs under the budget (width m, depth d, capacity c). *Proof sketch:* on the event $S_{\text{future}} \geq L$, the combined sufficient condition of the previous subsection holds for that window. When rankings are read from tracker counts, the separation argument is deterministic given retention, since SpaceSaving errors are one-sided and bounded by $W/(c{+}1)$ without randomness; ranking by Count-Min estimates instead pays an additional failure probability of at most e^{-d} per queried candidate, handled by a union bound and driven down geometrically in the depth d. The guarantee is marginal over future windows; it does not imply conditional validity for every traffic regime or time period.

Because drift breaks exchangeability, we also run adaptive conformal inference [13], which adjusts α_t online; its long-run tracking is empirical, not a finite-sample guarantee. Unlike conformal frequency estimation over sketches [14], which calibrates per-query frequency intervals, the conformalized object here is the *identifiability decision itself*.

Algorithm 1 (conformal budget pricing). The screen inverts into a pricing method: given L, target level α, depth d, and per-entry costs, output the memory-minimal budget that certifies future-window top-k identifiability, or *impossible* when $L = 0$. For fixed width m the minimal capacity is closed-form, $c_{\min}(m) = \lceil 1/(L/2 - e/m) \rceil - 1$, valid when $m > 2e/L$; pricing is therefore a one-dimensional search minimizing $M(m, c) = d\, m\, b_{\text{ctr}} + c\, b_{\text{cand}}$ over m (we use $b_{\text{ctr}} = 8$, $b_{\text{cand}} = 32$ bytes). The output is an exchangeability-qualified conformal price: the memory at which sketch-based top-k answers become certifiable for this stream, marginal over future windows.

Algorithm 2 (certified router). The components compose into one operational decision per stream, given a prefix, a memory budget, target k, and mode. The router (i) picks the exact path from two fallible signals – the shape heuristic and direct prefix profiling – committing only when they agree and abstaining to the safe sparse default otherwise; (ii) runs Algorithm 1 to classify sketching as *certifiable within budget*, *certifiable over budget*, *not worth it*, or *unidentifiable* ($L = 0$); (iii) picks GraphBLAS hypersparse when labels live natively in the 2^{32} space, compact SciPy CSR otherwise; and (iv) emits online and batch routes with reasons. Abstention is deliberate: the heuristic misses on CTU-13 and profiling misses on hotspot at full scale, but they disagree exactly there, and the safe default pays a bounded $1.28\times$ instead of a $2\times$ wrong-signal penalty.

V. Experiments

Experiments were run on a MacBook Air with Apple M3 CPU, 8 cores, 16 GB RAM, macOS 26.5.1 arm64, Python 3.9.6, NumPy 2.0.2 with Apple Accelerate BLAS/LAPACK, SciPy 1.13.1, pandas 2.3.3, matplotlib 3.9.4, and zstandard 0.25.0. No GPU, distributed runtime, or manual BLAS thread pinning is used.

The official-data experiments use the two organizer-generated products in the GraphChallenge S3 bucket – the Random PCAP (13.9 GB `pcap.zst`) and the Random GraphBLAS matrices (8.6 GB tar); the CAIDA-telescope-derived variant is access-controlled and not used. An HTTP Range request fetches the first 256 MiB (later 2 GiB) of the `pcap.zst`; the truncated zstd frame is stream-decompressed and 5,000,000 (respectively 40,000,000) fixed 40-byte RAW-IP packet records are parsed with a vectorized NumPy path in under five seconds per 5,000,000 packets. Identifiers are the challenge-provided anonymized addresses used as opaque integer labels. A provenance manifest records URL, byte range, parser path, counts, and output checksum. One 67 MB inner tar of the GraphBLAS product (the first 2^{23}-packet segment) is fetched the same way and deserialized with python-graphblas on SuiteSparse:GraphBLAS. The small benchmark uses 5,000–100,000-record edge tables with three repeats and three exact methods; the large benchmark uses 1,000,000 and 5,000,000 records, three repeats with the median reported, and two scalable methods. Runtimes exclude synthetic generation and disk I/O; the input table is memory-resident. The streaming benchmark uses 25,000–500,000 records; width and capacity sensitivity studies use 100,000. Selector and screen experiments use the first 50,000 records of each 5,000,000-edge stream; conformal experiments use 50,000-record windows (56–100 per stream) split half calibration, half test. The real capture is CTU-13 scenario 42 [16], used as a real-flow trace after relabeling addresses to opaque integer identifiers. Time-bin experiments inject four controlled anomaly types, and an official-format smoke test round-trips a Matrix Market matrix. Commands are exposed through the repository Makefile.

The key correctness check is output equivalence. For every benchmark row, the matrix produced by the method under test

TABLE I
LARGE BENCHMARK AT 5,000,000 INPUT EDGES (MEDIAN OF THREE REPEATS; STANDARD DEVIATION AT MOST 0.02 S PER CELL). RUNTIME EXCLUDES SYNTHETIC GENERATION AND DISK I/O.

Regime	nnz/edge	Sparse	Sp. M/s	pandas	pd. M/s
Hotspot	0.044	0.142	35.2	0.111	45.0
Community	0.451	0.285	17.5	0.591	8.5
Scanner	0.992	0.377	13.3	1.195	4.2
Uniform	0.999	0.366	13.7	1.206	4.1

is compared with the sparse direct matrix. Total traffic is also compared. A benchmark row is marked valid only when both checks pass.

VI. RESULTS

All numbers and figures in this section were generated on the machine described in Section V, and the repository Makefile regenerates every table and figure from saved CSV outputs.

Table I reports the 5,000,000-edge large benchmark. The regimes differ substantially: hotspot traffic has 222,097 nonzeros and high concentration, while uniform sparse traffic has 4,997,173 nonzeros and density 0.0011. Sparse direct construction is faster than pandas groupby in community, scanner, and uniform regimes. In the hotspot regime, pandas groupby is faster on this machine, which is a useful negative result: the best construction path depends on traffic shape and duplication rate. Throughput scales nearly linearly from 1,000,000 to 5,000,000 edges (sparse direct stays at roughly 13–35 million edges per second across regimes at both sizes), so the reported rates are not a small-input artifact. The table reports total measured construction plus analytics time and throughput for in-memory edge tables.

The streaming/sketch result at 500,000 edges (327,680-byte Count-Min, capacity 512): top-20 recall is 1.00 on hotspot at median relative error 0.0003, but 0.10 on community and 0 on scanner and uniform. A width sweep from 512 to 32,768 drives hotspot weight error to 0 yet recovers no missed diffuse heavy hitter at fixed capacity. Figure 1(b) shows the missing factor: capacity 64 to 8,192 lifts community recall from 0 to 1.00 and scanner from 0 to 0.95, and recall organizes cleanly along the identifiability index across both decoder families. With the lazy-deletion heap tracker this costs almost nothing at run time (0.62 s to 0.71–0.91 s at 100,000 edges): capacity is a memory knob, not a latency penalty. Streaming remains slower than SciPy construction in Python; its value is bounded-memory online processing.

The online method-selector experiment: from the first 50,000 records the selector matches the saved 5,000,000-edge exact-method winner in all four synthetic regimes and marks fixed-candidate sketching unsafe for the three diffuse ones (per-regime features in the repository CSVs) – but it misses on real traffic below, which is why certification, not the heuristic, is the primary contribution.

The plug-in safety screen of Section IV, on the same 50,000-record prefixes: under width 8,192 and capacity 512 no regime satisfies the sufficient conditions, while width 16,384 with capacity 5,000 (a 655 KB sketch) certifies hotspot with margin $+5.6 \times 10^{-5}$. Notably, the retention bound predicts the capacity scale the empirical sweep independently confirms: $c_{\min} \approx 2{,}000$ for community (recall 0.65 at 2,048, 1.00 at 8,192) and $c_{\min} \approx 5{,}800$ for scanner (0.95 only at 8,192). The screen tells an operator when sketch safety is not proven and what budget would prove it – under Assumption A1, which the conformal analysis below revisits.

A. *Official-Dataset Validation*

The candidate-capacity finding and the online diagnostics were checked on the official challenge data products. Table II summarizes the 5,000,000-packet prefix of the official Random PCAP – an extreme diffuse regime: every packet a unique pair, 4,997,115 unique anonymized sources, every packet a 40-byte header-only record with equal weights, consistent with the organizers' randomized generation. From the first 50,000 records the selector recommends sparse direct and flags fixed-candidate sketching unsafe; the measured end-to-end winner agrees (2.52 s versus 5.15 s pandas). The safety screen withholds certification because the top-k ranking gap is exactly zero. The capacity sweep confirms the diagnosis from the other direction: strict-identity top-20 recall is zero at every capacity, tie-aware recall is trivially 1.0, and IdentifiableRecall@20 reports *not identifiable* (empty core) – replacing a misleading zero with the correct statement that top-k identity carries no information in an all-tie regime. Unlike the synthetic diffuse regimes, where capacity rescues recall because heavy hitters exist, here sketch-based top-k is ill-posed. Eight disjoint 5,000,000-packet segments of the 40,000,000-packet fetch yield identical diagnoses.

Two further checks tie the toolkit to the official GraphBLAS ecosystem. First, an exact cross-check: the official product stores one $2^{32} \times 2^{32}$ UINT32 matrix per 2^{17}-packet window; deserializing the first 2^{23}-packet segment (64 matrices) and comparing coordinate sets against our pcap-parsed edge table matches exactly in 64 of 64 windows (8,388,608 pairs, after the bijective host/network byte-order relabeling) – our parser reproduces the official matrix pipeline without running the official code. Second, a SuiteSparse:GraphBLAS baseline on the same 5,000,000 edges, decomposed to avoid ambiguity: label factorization 2.1 s (shared by both compact paths; it dominates the end-to-end times in Table II); SciPy COO$\rightarrow$CSR kernel 0.106 s; GraphBLAS `from_coo` kernel 0.086–0.088 s; and GraphBLAS `from_coo` in the native $2^{32} \times 2^{32}$ hypersparse space, 0.086 s end to end with no factorization – an architectural advantage SciPy CSR cannot replicate, its row-pointer array alone needing tens of gigabytes.

Compatibility checks pass throughout: a Matrix Market round-trip exactly matches the edge-table matrix, and a local implementation of GraphChallenge-paper-style formulas agrees on core scalars for a 100,000-edge sample (repository CSVs) – not the full official reference repository, but a check that quantities agree before performance claims are interpreted.

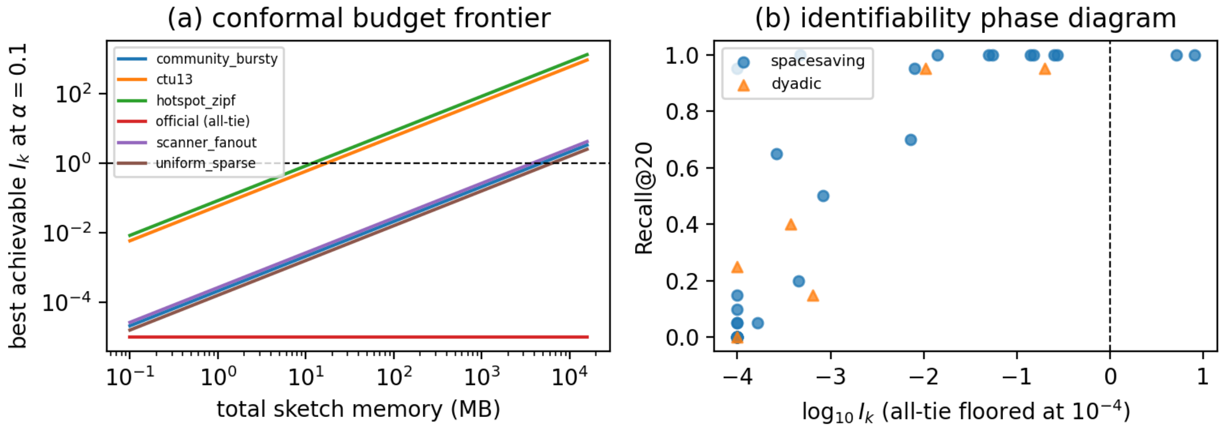

Fig. 1. (a) Conformal budget frontier: best achievable I_k versus total sketch memory per source; each curve crosses the certification line $I_k = 1$ at that stream's price from Algorithm 1 (hotspot 11.7 MB, CTU-13 16.8 MB, diffuse regimes 3.7–6.2 GB); the all-tie official product never crosses. (b) Identifiability phase diagram: measured Recall@20 versus $\log_{10} I_k$ for SpaceSaving tracking and dyadic descent across all sources and budgets, including runs at the priced budgets. The soundness claim is one-sided: no certified point ($I_k \geq 1$) shows low recall, while high-recall points left of the line reflect the screen's deliberate conservatism. I_k for the dyadic decoder maps its beam to the capacity term and is approximate.

TABLE II

OFFICIAL RANDOM PCAP PREFIX EXPERIMENT (5,000,000 PACKETS PARSED FROM A 256 MIB BYTE-RANGE REQUEST IN UNDER FIVE SECONDS; CONSTRUCTION TIMES ARE THE MINIMUM OF THREE REPEATS).

Quantity	Value
Packets parsed (unique pairs)	5,000,000 (all)
Sparse / pandas end-to-end (s)	2.52 / 5.15
Winner, selector recommendation	sparse_direct (match)
Safety-screen decision	not certified (gap = 0)
Recall@20 strict / tie-aware	0.00 / 1.00
Identifiable core H_{20}^{γ}	empty (not identifiable)

B. Real Traffic, Conformal Screens, and Decoders

We add CTU-13 scenario 42 [16] (2,824,636 bidirectional flows, 697,518 unique pairs, duplication 0.75, addresses relabeled to opaque integers) and report three experiments closing the loop on A1 and the sparse-recovery view.

First, honesty about the heuristic selector: on CTU-13 its concentration features (edge Gini 0.98) trigger the pandas recommendation, but sparse direct measures faster (0.087 s versus 0.175 s) – the selector's first miss. This negative result is useful: it separates heuristic method selection from certified identifiability, and Algorithm 2 converts it into abstention rather than error. The sketch-safety side is unaffected: top-20 recall is 0.95 at capacity 64 and 1.00 from 512 up, at median relative error 1.6×10^{-4}. Under the budget-matched metric, CTU-13's identifiable core holds 14 of the top 20 and hotspot's holds 10, all recovered (IdentifiableRecall 1.00); the diffuse regimes have empty cores at the default budget, consistent with the screen by construction.

Second, conformal screens and budget pricing (Table III, Figure 1(a)). Calibrating on the first half of each stream's 50,000-record windows and testing on the rest shows why one prefix should not be trusted: the hotspot window-level gap varies by two orders of magnitude, fixed calibration undercovers on drifting streams (empirical coverage as low as 0.82 at target 0.90), and adaptive conformal inference restores realized miscoverage to 0.10–0.11 on every tested source. The conformal screen consequently *refuses* the 655 KB budget the plug-in screen had certified for hotspot – the first window was optimistic. Algorithm 1 then prices exchangeability-qualified certification: 11.7 MB for hotspot, 16.8 MB for CTU-13, 3.7–6.2 GB for the diffuse regimes (finite but far beyond sketching's useful range, so exact construction is the right tool there), and *impossible* for the all-tie official product. Running the tracker at the priced budgets yields measured Recall@20 of 1.00 on both hotspot and CTU-13, so the certified region is not vacuous. Removing A1 is not free; Algorithm 1 reports its memory price.

Third, decoder independence. Replacing per-record SpaceSaving tracking with the dyadic descent of Section IV reproduces the same identifiability gradient: recall (tracker/dyadic) is 1.00/0.95 on hotspot, 1.00/0.95 on CTU-13, degraded on the diffuse regimes (0.15/0.40, 0.05/0.25, 0.05/0.15), and exactly 0.00/0.00 on the all-tie official product. Discovery is bounded by signal structure, not data-structure choice, as the sparse-recovery view predicts.

Finally, Table IV shows Algorithm 2's end-to-end decisions at a 64 MB budget. The two abstentions are the signal-disagreement cases: on CTU-13 abstention avoids the heuristic's 2× mistake; on hotspot it pays a bounded 1.28×. Sketching is routed only where certifiable and worth its price; the all-tie official product is declared unidentifiable and served by exact construction on the hypersparse backend.

Finally, the time-bin capability: in all four controlled scenarios (volume burst, scanner fanout, concentrated destination, distributed low-rate scan), the injected bin is the top-ranked bin

TABLE III

CONFORMAL SCREENS AND ALGORITHM 1 PRICES AT $\alpha = 0.1$ OVER 50,000-RECORD WINDOWS. $L_{0.1}$ IS THE CONFORMAL LOWER BOUND ON THE FUTURE-WINDOW GAP SHARE; FIXED AND ACI ARE HELD-OUT EMPIRICAL MISCOVERAGE UNDER FIXED AND ADAPTIVE CALIBRATION (TARGET 0.10); 655KB ASKS WHETHER THE BUDGET THE PLUG-IN SCREEN CERTIFIED FOR HOTSPOT PASSES; PRICE IS ALGORITHM 1'S MEMORY-MINIMAL CERTIFIED BUDGET.

Source	Win.	$L_{0.1}$	Fixed	ACI	655KB	Price
Hotspot	100	4.4×10^{-5}	.18	.10	no	11.7 MB
CTU-13 (real)	56	3.1×10^{-5}	.11	.11	no	16.8 MB
Community	100	1.1×10^{-7}	.06	.10	no	4.7 GB
Scanner	100	1.4×10^{-7}	.16	.11	no	3.7 GB
Uniform	100	8.4×10^{-8}	.08	.10	no	6.2 GB
Official (all-tie)	100	0	.00	.00	no	impossible

TABLE IV

CERTIFIED-ROUTER OUTPUT (ALGORITHM 2) PER SOURCE: EXACT PATH FROM AGREEING SIGNALS OR ABSTENTION, SKETCH DECISION WITH THE ALGORITHM 1 PRICE, AND THE BATCH ROUTE. ONLINE ROUTES USE THE CERTIFIED SKETCH WHERE WITHIN BUDGET (64 MB) AND EXACT PER-WINDOW CONSTRUCTION OTHERWISE.

Source	Exact path	Sketch decision (price)	Batch route
Hotspot	sparse (abstain)	certifiable (11.7 MB)	sparse, CSR
CTU-13	sparse (abstain)	certifiable (16.8 MB)	sparse, CSR
Community	sparse (agree)	not worth it (4.7 GB)	sparse, CSR
Scanner	sparse (agree)	not worth it (3.7 GB)	sparse, CSR
Uniform	sparse (agree)	not worth it (6.2 GB)	sparse, CSR
Official	sparse (agree)	unidentifiable	sparse, hypersparse

by the robust anomaly score, and a different feature explains each event type. We report this as a toolkit capability rather than a core contribution; per-scenario curves and numbers are in the repository figures and CSVs.

VII. LIMITATIONS

This submission deliberately avoids overclaiming. The official-data experiments use byte-range prefixes (up to 40,000,000 packets), not a full 2^{30}-packet run; prefix composition may differ from the full product (the eight-segment check mitigates, not eliminates, this); and the Random products are organizer-generated synthetic traffic – the CAIDA-telescope variant is access-controlled and unused, so no claim is made about real telescope traffic. The conformal screens provide marginal, not conditional, coverage; validity rests on window exchangeability, which drift violates – fixed calibration then under-covers, and adaptive conformal inference restores target miscoverage only empirically; with 50–100 windows per stream, levels below $\alpha \approx 0.02$ are unresolvable. The real-traffic evidence is one CTU-13 scenario; runtimes exclude disk I/O and generation (fetch/parse times in the manifest); a same-machine official-reference comparison remains future work.

ARTIFACT AVAILABILITY

The complete repository – source, tests, Makefile, result CSVs, and provenance manifests with checksums behind every table and figure – is available at https://github.com/QifanChen62/graphsense-network-sensing; `make paper-numbers` regenerates every table and figure.

VIII. CONCLUSION

We presented a reproducible toolkit for anonymized network sensing whose core is certified, memory-priced identifiability, composed by a certified router into one operational decision per stream: a sufficient-condition safety screen with a SpaceSaving retention bound, a top-k identifiability index with a budget-matched recall metric, conformal identifiability screens, and a pricing algorithm that replaces the prefix-representativeness assumption with exchangeability-qualified marginal coverage at a measured memory price – 11.7 MB for a synthetic hotspot regime, 16.8 MB for real CTU-13 traffic (both with measured recall 1.0 at the priced budgets), gigabytes for diffuse regimes where exact construction is the right tool, and *impossible* for the all-tie official product. The central empirical result, explained by the sparse-recovery view and reproduced by two decoder families, is that candidate capacity rather than sketch width bounds heavy-hitter discovery on diffuse traffic, at the scale the retention bound predicts. Reported failure cases – a selector miss on real traffic, fixed-calibration under-coverage under drift – are part of the contribution: they make approximation risk visible and priced. The package is a bridge between official GraphChallenge materials and certified commodity-CPU experimentation.

REFERENCES

[1] "Anonymized network sensing graph challenge," arXiv:2409.08115, 2024.

[2] "MIT/IEEE/Amazon GraphChallenge," https://graphchallenge.mit.edu/, accessed 2026-06-29.

[3] "GraphChallenge Submission Instructions," https://graphchallenge.mit.edu/submit, accessed 2026-06-29.

[4] A. Buluç, T. Mattson, S. McMillan, J. Moreira, and C. Yang, "The graphblas c api specification," in *IEEE International Parallel and Distributed Processing Symposium Workshops*, 2017.

[5] T. A. Davis, "Algorithm 1000: Suitesparse:graphblas: Graph algorithms in the language of sparse linear algebra," *ACM Transactions on Mathematical Software*, 2019.

[6] I. Voloshchuk, H. Jananthan, C. Byun, and J. Kepner, "Improving the graph challenge reference implementation," arXiv:2601.00974, 2026.

[7] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *Journal of Algorithms*, 2005.

[8] J. Misra and D. Gries, "Finding repeated elements," *Science of Computer Programming*, 1982.

[9] A. Metwally, D. Agrawal, and A. El Abbadi, "Efficient computation of frequent and top-k elements in data streams," in *International Conference on Database Theory*, 2005.

[10] A. Kumar, J. Xu, J. Wang, O. Spatscheck, and L. Li, "Sketching data streams for network measurements," in *Proceedings of the ACM SIGCOMM Conference*, 2004.

[11] A. Gilbert and P. Indyk, "Sparse recovery using sparse matrices," *Proceedings of the IEEE*, vol. 98, no. 6, 2010.

[12] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*. Springer, 2005.

[13] I. Gibbs and E. Candès, "Adaptive conformal inference under distribution shift," in *Advances in Neural Information Processing Systems*, 2021.

[14] M. Sesia and S. Favaro, "Conformal frequency estimation with sketched data," in *Advances in Neural Information Processing Systems*, 2022.

[15] A. Lakhina, M. Crovella, and C. Diot, "Mining anomalies using traffic feature distributions," in *Proceedings of the ACM SIGCOMM Conference*, 2005.

[16] S. García, M. Grill, J. Stiborek, and A. Zunino, "An empirical comparison of botnet detection methods," *Computers & Security*, vol. 45, 2014.